

PARTICAL-3

CODETANTRA

Home Learn Anywhere

202501050029@mitaoe.ac.in Support Logout

3.2.7. Student Data Analysis and Operations

Write a Python program that takes the file name of a CSV file containing student details, including roll numbers and their marks in three subjects as input, reads the data, and performs the following operations:

- Print all student details: Display the complete details of all students, including roll numbers and marks for all subjects.
- Find total students: Determine the total number of students in the dataset.
- Print all student roll numbers: Extract and print the roll numbers of all students.
- Print Subject 1 marks: Extract and print the marks of all students in Subject 1.
- Find minimum marks in Subject 2: Identify the lowest marks in Subject 2.
- Find maximum marks in Subject 3: Identify the highest marks in Subject 3.
- Print all subject marks: Display the marks of all students for each subject.
- Find total marks of students: Compute the total marks for each student across all subjects.
- Find the average marks of each student: Compute the average marks for each student.
- Find average marks of each subject: Compute the average marks for all students in each subject.
- Find average marks of Subject 1 and Subject 2: Compute the average marks for Subject 1 and Subject 2.
- Find average marks of Subject 1 and Subject 3: Compute the average marks for Subject 1 and Subject 3.
- Find the roll number of the student with maximum marks in Subject 3: Identify the student with the highest marks in Subject 3 and print their roll number.
- Find the roll number of the student with minimum marks in Subject 2: Identify the student with the lowest marks in Subject 2 and print their roll number.
- Find the roll number of students who scored 24 marks in Subject 2: Identify students who obtained exactly 24 marks in Subject 2 and print their roll numbers.
- Find the count of students who got less than 40 marks in Subject 4: Count the number of

Operation...

```
1 import numpy as np
2
3 a = np.loadtxt("Sample.csv", delimiter=',', skiprows=1)
4
5 # 1. Print all student details
6 print("All student Details:\n", a)
7
8 # 2. print total students
9
10 print("Total Students:", a.shape[0])
11
12 # 3. Print all student Roll numbers
13 print("All Student Roll Nos", a[:, 0])
14
15 # 4. Print subject 1 marks
16 print("Subject 1 Marks", a[:, 1])
17
18 # 5. print minimum marks of Subject 2
19 print("Min marks in Subject 2", np.min(a[:, 2]))
20
21 # 6. print maximum marks of Subject 3
22 print("Max marks in Subject 3", np.max(a[:, 3]))
23
24 # 7. Print All subject marks
25 print("All subject marks:", a[:, 1:])
```

CODETANTRA

Home Learn Anywhere

202501050029@mitaoe.ac.in Support Logout

3.2.6. Numpy: Searching, Sorting, Counting, Broadcasting

The given code in the editor takes a single array, array1, as space-separated integers as input from the user.
Additionally, it takes the following inputs:

- search_value: The value to search for in the array.
- count_value: The value to count its occurrences in the array.
- broadcast_value: The value to add for broadcasting across the array.

You need to complete the code to perform the following operations:

- Searching: Find the indices where search_value appears in array1 and print these indices.
- Counting: Count how many times count_value appears in array1 and print the count.
- Broadcasting: Add broadcast_value to each element of array1 using broadcasting, and print the resulting array.
- Sorting: Sort array1 in ascending order and print the sorted array.

Input Format:

- A single line containing space-separated integers representing array1.
- An integer search_value represents the value to search for in the array.
- An integer count_value represents the value to count in the array.
- An integer broadcast_value represents the value to add to each element of the array.

Output Format:

- The indices where search_value occurs in array1.
- The count of occurrences of count_value in array1.
- The array after adding the broadcast_value to each element.

arrayOpe...

```
1 import numpy as np
2
3 # Input array from the user
4 array1 = np.array(list(map(int, input().split())))
5
6 # Searching
7 search_value = int(input("Value to search: "))
8 count_value = int(input("Value to count: "))
9 broadcast_value = int(input("Value to add: "))
10
11 # Find indices where value matches in array1
12 a=np.where(array1==search_value)[0]
13 print(a)
14 # Count occurrences in array1
15 b=np.count_nonzero(array1==count_value)
16 print(b)
17 # Broadcasting addition
18 c=array1 + broadcast_value
19 print(c)
20 # Sort the first array
21 d=np.sort(array1)
22 print(d)
```

CODETANTRAHomeLearn Anywhere

202501050029@mitaoe.ac.inSupportLogout

3.2.5. Numpy: Copying and Viewing Arrays

1031

🔍🌙🔗🔗🔗

The given code takes a list of integers as input and converts it into a NumPy array. Your task is to complete the code by:

- Creating a view of the original_array and assigning it to view_array.
- Creating a copy of the original_array and assigning it to copy_array.

After completing these steps, observe how modifying the view affects the original_array, while modifying the copy does not.

Input Format:

- A single line of space-separated integers.

Output Format:

- After modifying the view:

Original array after modifying view: <original_array>
View array: <view_array>
- After modifying the copy:

Original array after modifying copy: <original_array>
Copy array: <copy_array>

Sample Test Cases

+

copyAnd...

1import numpy as np

2

3inputlist = list(map(int,input().split(" ")))

4

5# Original array

6original_array = np.array(inputlist)

7

8# Create a view

9view_array =original_array.view()

10

11# Create a copy

12copy_array =original_array.copy()

13

14# Modify the view

15view_array[0] = 99

16print("Original array after modifying view:", original_array)

17print("View array:", view_array)

18

19# Modify the copy

20copy_array[1] = 88

21print("Original array after modifying copy:", original_array)

22print("Copy array:", copy_array)

23

TerminalTest cases

< PrevResetSubmitNext >

CODETANTRAHomeLearn Anywhere

202501050029@mitaoe.ac.inSupportLogout

3.2.4. Numpy: Arithmetic and Statistical Operations, Mathematical Operations, Bitw...

0001

🔍🌙🔗🔗🔗

You are given two arrays A and B. Your task is to complete the function array_operations, which will convert these lists into NumPy arrays and perform the following operations:

1. Arithmetic Operations:

- Compute the element-wise sum, difference, and product of the two arrays.

2. Statistical Operations:

- Calculate the mean, median, and standard deviation of array A.

3. Bitwise Operations:

- Perform bitwise AND, bitwise OR, and bitwise XOR on the arrays (ex: A₁ OR B₁).

Input Format:

- The first line contains space-separated integers representing the elements of array A.
- The second line contains space-separated integers representing the elements of array B.

Output Format:

- For each operation (arithmetic, statistical, and bitwise), print the results in the specified format as shown in sample test cases.

Sample Test Cases

+

different...

15median_A = np.median(A)

16std_dev_A = np.std(A)

17

18# Bitwise Operations

19and_result = A & B

20or_result = A | B

21xor_result = A ^ B

22

23# Output results with one space between each element

24print("Element-wise Sum:", ' '.join(map(str, sum_result)))

25print("Element-wise Difference:", ' '.join(map(str, diff_result)))

26print("Element-wise Product:", ' '.join(map(str, prod_result)))

27

28print(f"Mean of A: {mean_A}")

29print(f"Median of A: {median_A}")

30print(f"Standard Deviation of A: {std_dev_A}")

31

32print("Bitwise AND:", ' '.join(map(str, and_result)))

33print("Bitwise OR:", ' '.join(map(str, or_result)))

34print("Bitwise XOR:", ' '.join(map(str, xor_result)))

35

36A = list(map(int, input().split())) # Elements of array A

37B = list(map(int, input().split())) # Elements of array B

38array_operations(A, B)

39

TerminalTest cases

< PrevResetSubmitNext >

CODETANTRAHomeLearn Anywhere

202501050029@mitaoe.ac.inSupportLogout

3.2.4. Numpy: Arithmetic and Statistical Operations, Mathematical Operations, Bitw...

0001

🔍🌙🔗🔗🔗

You are given two arrays A and B. Your task is to complete the function array_operations, which will convert these lists into NumPy arrays and perform the following operations:

1. Arithmetic Operations:

- Compute the element-wise sum, difference, and product of the two arrays.

2. Statistical Operations:

- Calculate the mean, median, and standard deviation of array A.

3. Bitwise Operations:

- Perform bitwise AND, bitwise OR, and bitwise XOR on the arrays (ex: A₁ OR B₁).

Input Format:

- The first line contains space-separated integers representing the elements of array A.
- The second line contains space-separated integers representing the elements of array B.

Output Format:

- For each operation (arithmetic, statistical, and bitwise), print the results in the specified format as shown in sample test cases.

Sample Test Cases

+

different...

1import numpy as np

2

3def array_operations(A, B):

4

5# Convert A and B to NumPy arrays

6A= np.array(A)

7B= np.array(B)

8

9# Arithmetic Operations

10sum_result = A + B

11diff_result = A - B

12prod_result = A * B

13

14# Statistical Operations

15mean_A = np.mean(A)

16median_A = np.median(A)

17std_dev_A = np.std(A)

18

19# Bitwise Operations

20and_result = A & B

21or_result = A | B

22xor_result = A ^ B

23

24# Output results with one space between each element

25print("Element-wise Sum:", ' '.join(map(str, sum_result)))

26print("Element-wise Difference:", ' '.join(map(str, diff_result)))

27

TerminalTest cases

< PrevResetSubmitNext >

CODETANTRAHomeLearn Anywhere

202501050029@mitaoe.ac.inSupportLogout

3.2.3. Numpy: Custom Sequence Generation

Write a Python program that takes the following inputs from the user:

- Start value: The starting point of the sequence.
- Stop value: The sequence should end before this value.
- Step value: The increment between each number in the sequence.

The program should then generate a sequence using `numpy` based on these inputs and print the generated sequence.

Input Format:

- The user will input three integer values: start, stop, and step, each on a new line.

Output Format:

- The program should print the generated sequence based on the input values.

Sample Test Cases

customS...

```
1 import numpy as np
2
3 # Take user input for the start, stop, and step of the sequence
4 start = int(input())
5 stop = int(input())
6 step = int(input())
7
8 # Generate the sequence using np.arange()
9 a = np.arange(start, stop, step)
10 print(a)
11 # Print the generated sequence
12
```

TerminalTest cases

PrevResetSubmitNext

CODETANTRAHomeLearn Anywhere

202501050029@mitaoe.ac.inSupportLogout

3.2.2. Numpy: Horizontal and Vertical Stacking of Arrays

You are given two arrays `arr1` and `arr2`. You need to perform horizontal and vertical stacking operations on them using NumPy.

- Horizontal Stacking:** Stack the two matrices horizontally (side by side).
- Vertical Stacking:** Stack the two matrices vertically (one below the other).

Input Format:

- The program should first prompt the user to input two 3x3 arrays.
- Each array consists of 3 rows, and each row contains 3 space-separated integers.
- The user will input the two arrays row by row.

Output Format:

- The program should display the result of the Horizontal Stack (side-by-side stacking) of the two arrays.
- The program should then display the result of the Vertical Stack (one below the other) of the two arrays.

Sample Test Cases

stacking.py

```
1 import numpy as np
2
3 # Input matrices
4 print("Enter Array1:")
5 arr1 = np.array([list(map(int, input().split())) for i in range(3)])
6
7 print("Enter Array2:")
8 arr2 = np.array([list(map(int, input().split())) for i in range(3)])
9
10 # Perform horizontal stacking (hstack)
11 a = np.hstack((arr1, arr2))
12 print("Horizontal Stack:")
13 print(a)
14
15 # Perform vertical stacking (vstack)
16 b = np.vstack((arr1, arr2))
17 print("Vertical Stack:")
18 print(b)
19
20
```

TerminalTest cases

PrevResetSubmitNext

CODETANTRAHomeLearn Anywhere

202501050029@mitaoe.ac.inSupportLogout

3.2.1. Numpy: Matrix Operations

The given code takes two 3×3 matrices, `matrix_a`, and `matrix_b`, as input from the user and converts them into NumPy arrays.

Task:

You are required to compute and display the results of the following matrix operations:

- Addition** (`matrix_a + matrix_b`)
- Subtraction** (`matrix_a - matrix_b`)
- Element-wise Multiplication** (`matrix_a * matrix_b`)
- Matrix Multiplication** (`matrix_a . matrix_b`)
- Transpose of Matrix A**

Input Format:

- The user will input 3 rows for `matrix_a`, each containing 3 integers separated by spaces.
- Similarly, the user will input 3 rows for `matrix_b`, each containing 3 integers separated by spaces.

Output Format:

The program should display the results of the operations in the following order:

- The result of Addition.
- The result of Subtraction.
- The result of Element-wise Multiplication.
- The result of Matrix Multiplication.
- The Transpose of Matrix A.

matrixOp...

```
9
10
11 # Addition
12 addition_result = matrix_a + matrix_b
13 print("Addition (A + B):")
14 print(addition_result)
15
16 # Subtraction
17 subtraction_result = matrix_a - matrix_b
18 print("Subtraction (A - B):")
19 print(subtraction_result)
20
21 # Multiplication (element-wise)
22 elementwise_multiplication_result = matrix_a * matrix_b
23 print("Element-wise Multiplication (A * B):")
24 print(elementwise_multiplication_result)
25
26 # Matrix multiplication (dot product)
27 matrix_multiplication_result = np.dot(matrix_a, matrix_b)
28 print("A dot B:")
29 print(matrix_multiplication_result)
30 # Transpose
31 transpose_a = matrix_a.T
32 print("Transpose of A:")
33 print(transpose_a)
```

TerminalTest cases

PrevResetSubmitNext

CODETANTRA

Home

Learn Anywhere

202501050029@mitaoe.ac.in

Support

Logout

3.2.1. Numpy: Matrix Operations

1130

Task:
The given code takes two 3×3 matrices, `matrix_a`, and `matrix_b`, as input from the user and converts them into NumPy arrays.

You are required to compute and display the results of the following matrix operations:
1. Addition (`matrix_a + matrix_b`)
2. Subtraction (`matrix_a - matrix_b`)
3. Element-wise Multiplication (`matrix_a * matrix_b`)
4. Matrix Multiplication (`matrix_a . matrix_b`)
5. Transpose of Matrix A

Input Format:
• The user will input 3 rows for `matrix_a`, each containing 3 integers separated by spaces.
• Similarly, the user will input 3 rows for `matrix_b`, each containing 3 integers separated by spaces.

Output Format:
The program should display the results of the operations in the following order:
1. The result of Addition.
2. The result of Subtraction.
3. The result of Element-wise Multiplication.
4. The result of Matrix Multiplication.
5. The Transpose of Matrix A.

matrixOp...

1 import numpy as np
2
3 # Input matrices
4 print("Enter Matrix A:")
5 matrix_a = np.array([list(map(int, input().split())) for i in range(3)])
6
7 print("Enter Matrix B:")
8 matrix_b = np.array([list(map(int, input().split())) for i in range(3)])
9
10
11 # Addition
12 addition_result = matrix_a + matrix_b
13 print("Addition (A + B):")
14 print(addition_result)
15
16 # Subtraction
17 subtraction_result = matrix_a - matrix_b
18 print("Subtraction (A - B):")
19 print(subtraction_result)
20
21 # Multiplication (element-wise)
22 elementwise_multiplication_result = matrix_a * matrix_b
23 print("Element-wise Multiplication (A * B):")
24 print(elementwise_multiplication_result)
25

Terminal

Test cases

CODETANTRA

Home

Learn Anywhere

202501050029@mitaoe.ac.in

Support

Logout

3.1.1. Numpy Array Operations

2223

Write a Python program to create a NumPy array based on user input and display the following:
• The created NumPy array.
• The number of dimensions of the array using `ndim`
• The shape of the array using `shape`
• The total number of elements in the array using `size`
Assume all input elements are valid integers.

Input Format:
• The first line contains two space-separated integers, `r` and `c`, representing the number of rows and the number of columns.
• The next `r` lines contain `c` space-separated integers representing the elements of the array, entered row by row.

Output Format:
• The created NumPy array (printed as a 2D array).
• An integer representing the number of dimensions of the array.
• A tuple representing the shape of the array.
• An integer representing the total number of elements in the array.

Note: Use `reshape()` function to reshape the input array with the specified number of rows and columns.

Sample Test Cases

numpyarr...

1 import numpy as np
2 rows, cols = map(int, input().split())
3
4 elements = []
5 for _ in range(rows):
6 elements.extend(map(int, input().split()))
7
8 array = np.array(elements).reshape(rows, cols)
9 print(array)
10 print(array.ndim)
11 print(array.shape)
12 print(array.size)
13 # write your code here...
14

Terminal

Test cases

CODETANTRA

Home

Learn Anywhere

202501050029@mitaoe.ac.in

Support

Logout

3.2.7. Student Data Analysis and Operations

3203

Write a Python program that takes the file name of a CSV file containing student details, including roll numbers and their marks in three subjects as input, reads the data, and performs the following operations:
• Print all student details: Display the complete details of all students, including roll numbers and marks for all subjects.
• Find total students: Determine the total number of students in the dataset.
• Print all student roll numbers: Extract and print the roll numbers of all students.
• Print Subject 1 marks: Extract and print the marks of all students in Subject 1.
• Find minimum marks in Subject 2: Identify the lowest marks in Subject 2.
• Find maximum marks in Subject 3: Identify the highest marks in Subject 3.
• Print all subject marks: Display the marks of all students for each subject.
• Find total marks of students: Compute the total marks for each student across all subjects.
• Find the average marks of each student: Compute the average marks for each student.
• Find average marks of Subject 1 and Subject 2: Compute the average marks for Subject 1 and Subject 2.
• Find average marks of Subject 1 and Subject 3: Compute the average marks for Subject 1 and Subject 3.
• Find the roll number of the student with maximum marks in Subject 3: Identify the student with the highest marks in Subject 3 and print their roll number.
• Find the roll number of the student with minimum marks in Subject 2: Identify the student with the lowest marks in Subject 2 and print their roll number.
• Find the roll number of students who scored 24 marks in Subject 2: Identify students who obtained exactly 24 marks in Subject 2 and print their roll numbers.
• Find the count of students who got less than 40 marks in Subject 1: Count the number of students who scored less than 40 marks in Subject 1.
• Find the count of students who got more than 90 marks in Subject 2: Count the number of students who scored more than 90 marks in Subject 2.
• Find the count of students who scored >=90 in each subject: Count the number of students who scored 90 or more marks in each subject.
• Find the count of subjects in which each student scored >=90: Determine how many subjects each student scored 90 or more marks in.

Operation...

25 print("All subject marks:", a[:,1:]-)
26
27 # 8. print Total marks of students
28 print("Total Marks", np.sum(a[:,1:],axis=1))
29
30 # 9. print average marks of each student
31 avg = np.mean(a[:,1:],axis=1)
32 print(np.round(avg,1))
33
34 # 10. print average marks of each subject
35 print("Average Marks of each subject",np.mean(a[:,1:],axis=0)-----)
36
37 # 11. print average marks of S1 and S2
38 print("Average Marks of S1 and S2", np.mean(a[:,1:3],axis=0)-----)
39
40 # 12. print average marks of S1 and S3
41 print("Average Marks of S1 and S3",np.mean(a[:,1,3],axis=0)-----)
42
43 # 13. print Roll number who got maximum marks in Subject 3
44
45 print("Roll no who got maximum marks in Subject 3", a[np.argmax(a[:, 3]),0]---)
46
47 # 14. print Roll number who got minimum marks in Subject 2
48
49 print("Roll no who got minimum marks in Subject 2", a[np.argmin(a[:,2]),0]----)
50
51 # 15. print Roll number who got 24 marks in Subject 2
52
53 print("Roll no who got 24 marks in Subject 2",--a[a[:,2]==24,0:1]---)
54

Terminal

Test cases